JavaScript en el Navegador, Asincronía y Consumo de APIs con Fetch

16 de septiembre de 2026

Resumen

El presente reporte de investigación describe los fundamentos del uso de JavaScript dentro del navegador, el modelo de asincronía basado en Promesas y la sintaxis `async/await`, así como el consumo de *endpoints* HTTP mediante la Fetch API. Se incluyen ejemplos prácticos de peticiones con método `GET` y `POST`, haciendo énfasis en el manejo de errores y en las buenas prácticas recomendadas por la documentación oficial.

Índice

1. Fundamentos de JavaScript en el Navegador **2**
1.1. Conexión de un script de JavaScript a HTML . . . 2
1.2. Manipulación básica del DOM . . . 2

2. Promesas y asincronía **3**
2.1. Promesas (Promises) . . . 3
2.2. Fetch API . . . 3
2.3. Sintaxis `async` / `await` . . . 3

3. Consumo de endpoints con `fetch()` **3**
3.1. Petición con método GET . . . 3
3.2. Petición con método POST . . . 4

4. Conclusión **5**

Referencias **5**

1. Fundamentos de JavaScript en el Navegador

1.1. Conexión de un script de JavaScript a HTML

Para ejecutar lógica de JavaScript dentro de una página web se utiliza la etiqueta HTML `<script>`. La forma recomendada es vincular un archivo externo mediante el atributo `src`:

```
<!-- Opcion A: incluir antes del cierre del body -->
<script src="app.js"></script>

<!-- Opcion B: incluir en el head con el atributo defer -->
<head>
    <script src="app.js" defer></script>
</head>
```

Listing 1: Vinculación de archivo JavaScript externo

Se recomienda colocar esta etiqueta justo antes de cerrar el elemento `</body>`, o bien incluir el atributo `defer` en el `<head>`. En ambos casos se garantiza que todo el documento HTML (el DOM) termine de cargarse en memoria antes de que el código JavaScript empiece a ejecutarse.

1.2. Manipulación básica del DOM

El **DOM** (*Document Object Model*) es la representación estructurada en forma de árbol de todos los elementos HTML de la página web. No forma parte del lenguaje JavaScript en sí, sino que es una **API del navegador** que JavaScript utiliza para interactuar dinámicamente con los elementos. Dos métodos fundamentales son:

- `document.getElementById()`: selecciona un elemento HTML único a partir de su atributo de identificación (`id`).
- `addEventListener()`: registra un "escuchador" (listener) de eventos sobre un elemento seleccionado, ejecutando una función *callback* cuando el usuario realiza una acción específica (como hacer clic o escribir).

```
// 1. Seleccionar elementos del DOM por su ID
const boton  = document.getElementById('miBoton');
const titulo = document.getElementById('miTitulo');

// 2. Escuchar el evento de clic en el boton
boton.addEventListener('click', () => {
    // Cambiar el texto del titulo dinamicamente
    titulo.textContent = 'El texto ha sido cambiado!';
});
```

Listing 2: Ejemplo de selección de elementos y manejo de eventos

2. Promesas y asincronía

2.1. Promesas (Promises)

Una **Promesa** en JavaScript es un objeto que representa el resultado eventual (exitoso o fallido) de una operación asíncrona. Permite manejar tareas que toman tiempo en completarse (como consultar una API o leer un archivo) sin congelar el hilo principal de ejecución de la interfaz web. Una promesa puede encontrarse en uno de tres estados:

- **Pending** (pendiente): estado inicial; la operación aún no se ha completado.
- **Fulfilled** (cumplida): la operación se completó con éxito y devuelve un valor.
- **Rejected** (rechazada): la operación falló y devuelve la razón o error.

2.2. Fetch API

La **Fetch API** es una interfaz moderna integrada nativamente en los navegadores web que proporciona el método `fetch()` para realizar peticiones HTTP de forma asíncrona. Reemplazó al antiguo objeto `XMLHttpRequest` y trabaja internamente mediante Promesas, lo que la hace más limpia y composable que su predecesor.

2.3. Sintaxis `async` / `await`

La sintaxis `async/await` es una mejora sintáctica (*syntactic sugar*) introducida en JavaScript para trabajar con Promesas de forma más limpia, permitiendo escribir código asíncrono que se lee casi como si fuera síncrono o secuencial:

- `async`: se coloca antes de la declaración de una función para indicar que siempre devolverá una promesa.
- `await`: se utiliza dentro de funciones `async` para pausar la ejecución del código hasta que la promesa se resuelva o se rechace.

3. Consumo de endpoints con `fetch()`

3.1. Petición con método GET

Se utiliza para consultar información de un servidor. Por defecto, la función `fetch()` realiza peticiones con método `GET` cuando solo se le proporciona la URL:

```
1 async function obtenerUsuarios() {
2     try {
3         // Realizar la peticion HTTP GET
4         const respuesta = await fetch('https://jsonplaceholder.
              typicode.com/users');
5
```

```
        // Verificar si la respuesta fue exitosa (status 200-299)
        if (!respuesta.ok) {
            throw new Error(`Error HTTP: ${respuesta.status}`);
        }

        // Convertir la respuesta a JSON (retorna una promesa)
        const usuarios = await respuesta.json();
        console.log('Lista de Usuarios:', usuarios);
    } catch (error) {
        console.error('Ocurrio un error al obtener datos:', error);
    }
}

obtenerUsuarios();
```

Listing 3: Ejemplo de petición GET usando async/await

3.2. Petición con método POST

Se utiliza para enviar datos nuevos hacia el servidor. Requiere pasar un segundo argumento (objeto de configuración) con los atributos `method`, `headers` indicando `'Content-Type': 'application/json'`, y el `body` convirtiendo el objeto JavaScript a texto mediante `JSON.stringify()`:

```
async function crearUsuario() {
    const nuevoUsuario = {
        name:  'Elena Rostova',
        email: 'elena@example.com'
    };

    try {
        const respuesta = await fetch('https://jsonplaceholder.
            typicode.com/users', {
            method: 'POST',                                    // Metodo
                HTTP
            headers: {
                'Content-Type': 'application/json'             //
                    Indicamos que enviamos JSON
            },
            body: JSON.stringify(nuevoUsuario)                 // Objeto
                JS -> cadena JSON
        });

        if (!respuesta.ok) {
            throw new Error(`Error al crear usuario: ${respuesta.
                status}`);
        }

        const usuarioCreado = await respuesta.json();
```

```
        console.log('Usuario creado exitosamente:', usuarioCreado);
    } catch (error) {
        console.error('Error en la peticion POST:', error);
    }
}

crearUsuario();
```

Listing 4: Ejemplo de petición POST usando async/await

> **Nota sobre el MIME type:** el tipo MIME correcto para enviar JSON es `application/json`. En algunas instrucciones aparece escrito como `application.json`, pero esta forma **no es un tipo MIME válido** y podría provocar que el servidor no interprete correctamente el cuerpo de la petición.

4. Conclusión

JavaScript, a través de las APIs del navegador, permite construir aplicaciones web dinámicas e interactivas. El modelo de asincronía basado en Promesas, junto con la sintaxis `async/await` y la Fetch API, ofrece una forma moderna, legible y robusta de consumir servicios HTTP. Comprender estos fundamentos es indispensable para el desarrollo de aplicaciones que dependen de datos externos, como las *Single Page Applications* y los clientes que consumen APIs REST.

Referencias

- MDN Web Docs. *Fetch API*. https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- MDN Web Docs. *Promise*. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise
- MDN Web Docs. *async function*. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function
- WHATWG. *DOM Living Standard*. https://dom.spec.whatwg.org/
- JSONPlaceholder. *Fake Online REST API*. https://jsonplaceholder.typicode.com/